Semantic Engineering

Dr. Kornelia Papp

Agenda

01	Session goals
02	Semantics
03	Exercise
04	Representation learning
05	Exercise

Session Goals

How we represent language determines what AI systems can compare, retrieve, cluster, and understand. The shift from word matching to embeddings is the shift from asking **“Do these texts use the same words?”** to **“Are they about similar things?”**

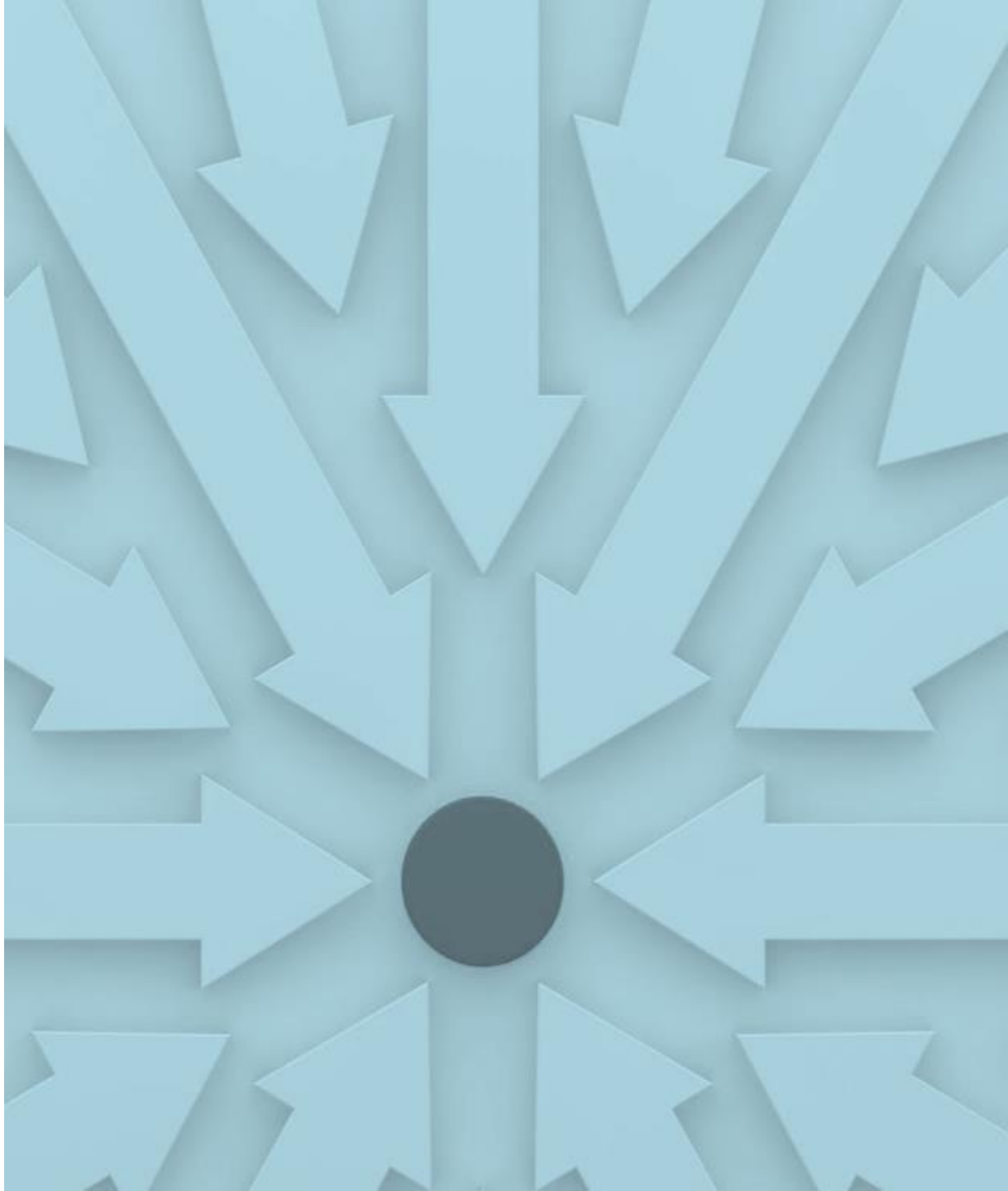
What is meaning?
Distinguish semantics, pragmatics, and context, and why the same words can mean different things.

What does “similar” mean?
Compare lexical, syntactic, and semantic similarity, and see why similarity always depends on the chosen definition.

How do machines compare text?
Understand distance metrics and cosine similarity as ways to compare numerical representations.

What are embeddings?
Explain how words, sentences, documents, and images can be represented as vectors in a meaningful space.

Semantic engineering is the bridge between human meaning and machine-readable representation.

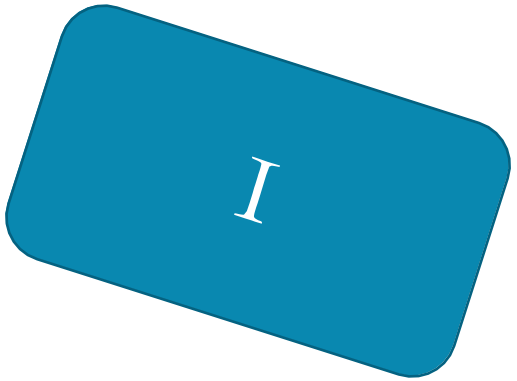


Agenda

01	Session goals
02	Semantics
03	Exercise
04	Representation learning
05	Exercise


you


than


I


chocolate

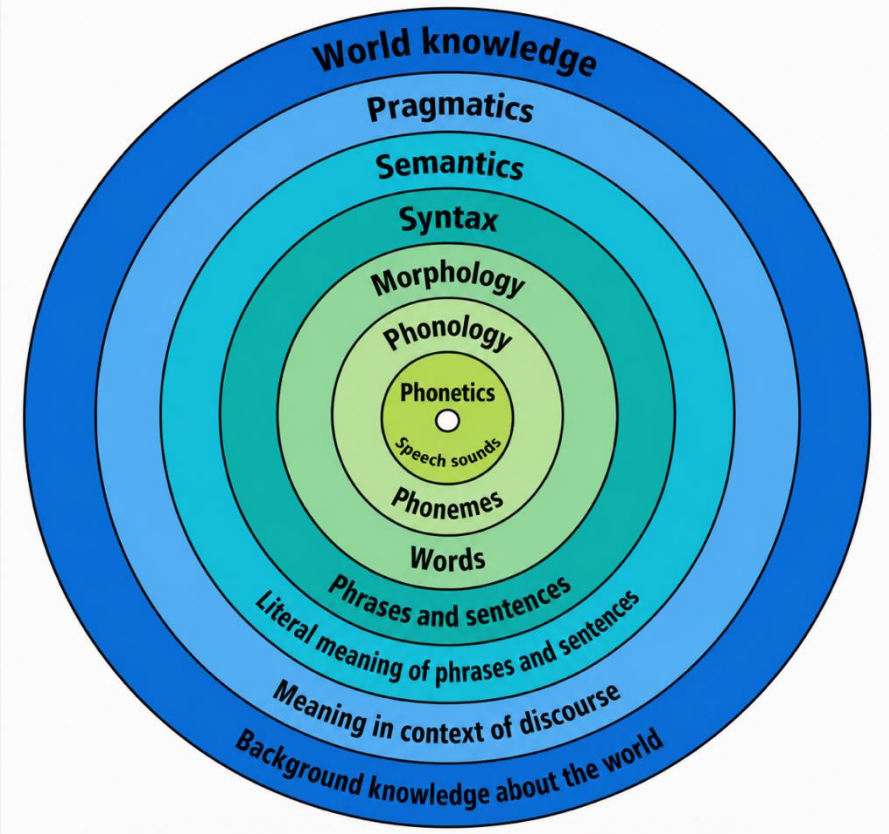

love

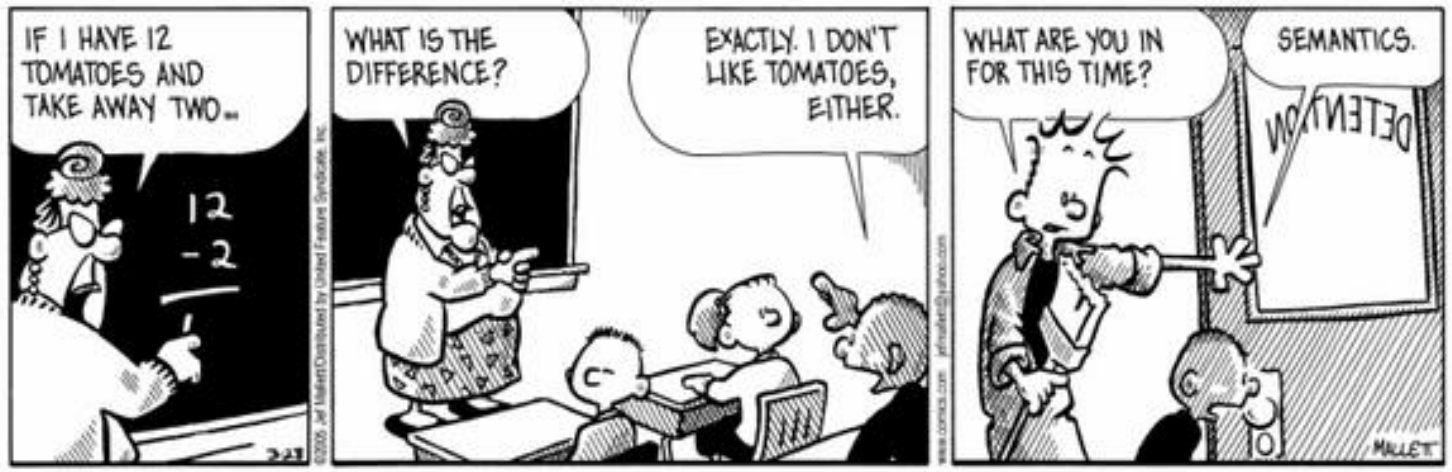

more


you love chocolate more than I
I love you more than chocolate
I love chocolate more than you
I love chocolate more than you

Semantics

Semantics is the process of understanding the meaning and interpretation of words, signs and sentence structure.





Pragmatics

A: Your greatest weakness?

B: Interpreting semantics of a question but ignoring the pragmatics

A: Could you give an example?

B: Yes, I could

Similarity Definitions

- Dictionary: the quality or state of being similar.
- Philosophy: the relation of sharing properties.
- Psychology: the psychological degree of identity of two mental representations.
- Biology: In biology, homology is similarity due to shared ancestry between a pair of structures or genes in different taxa.
- Geometry: two objects are similar if they have the same shape, or one has the same shape as the mirror image of the other.

“Lives are snowflakes - unique in detail, forming patterns we have seen before, but as like one another as peas in a pod (and have you ever looked at peas in a pod? I mean, really looked at them? There's not a chance you'd mistake one for another, after a minute's close inspection.)”

– Neil Gaiman

Similarity Definitions

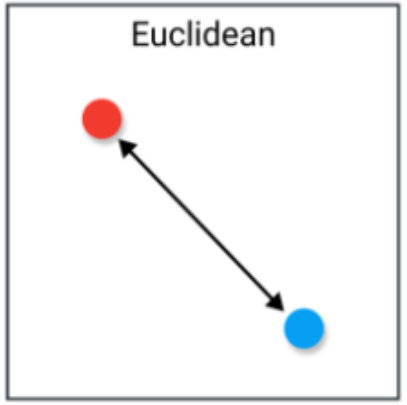
SIMILAR ⟷ DISSIMILAR

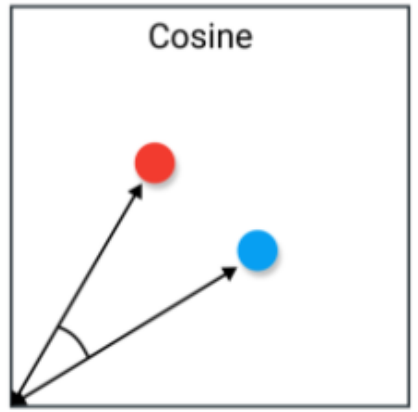
Similarity is Definition Dependent

The similarity between two strings can be quite different depending on how you define similarity in the first place.

- String similarity: ordered elements in the sequence
- Vector similarity: compares unordered sets

Although no single definition of a similarity measure exists, usually such measures are in some sense the inverse of distance metrics: they take on large values for similar objects and either zero or a negative value for very dissimilar objects. (Wikipedia)





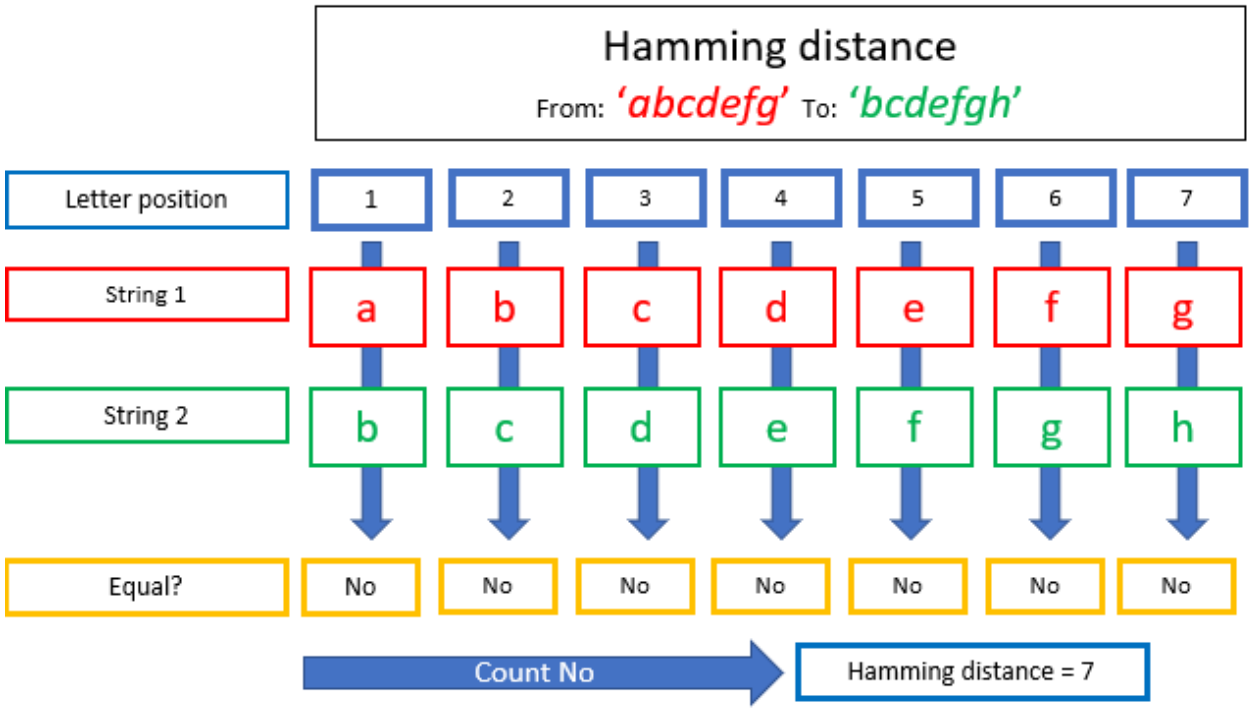
Types of Linguistic Similarity

SYNTACTICALLY SIMILAR:	I saw a castle on the hill.	~	I ate a cake at the table.
LEXICALLY SIMILAR:	I saw a castle on the hill.	~	I saw castles on the hill.
SEMANTICALLY SIMILAR:	I saw a castle on the hill.	~	There is a fort on the top of the hill.

Semantic similarity is a metric defined over a set of documents or terms, where the idea of distance between them is based on the likeness of their meaning or semantic content as opposed to similarity which can be estimated regarding their syntactical representation.

Hamming Distance

- String similarity
- Compares every letter of two strings
- Very fast
- Cannot compare strings of different length



https://towardsdatascience.com/3-text-distances-that-every-data-scientist-should-know-7fcdf850e510

Typical use cases include error correction when data is transmitted over computer networks.

Levenshtein Distance

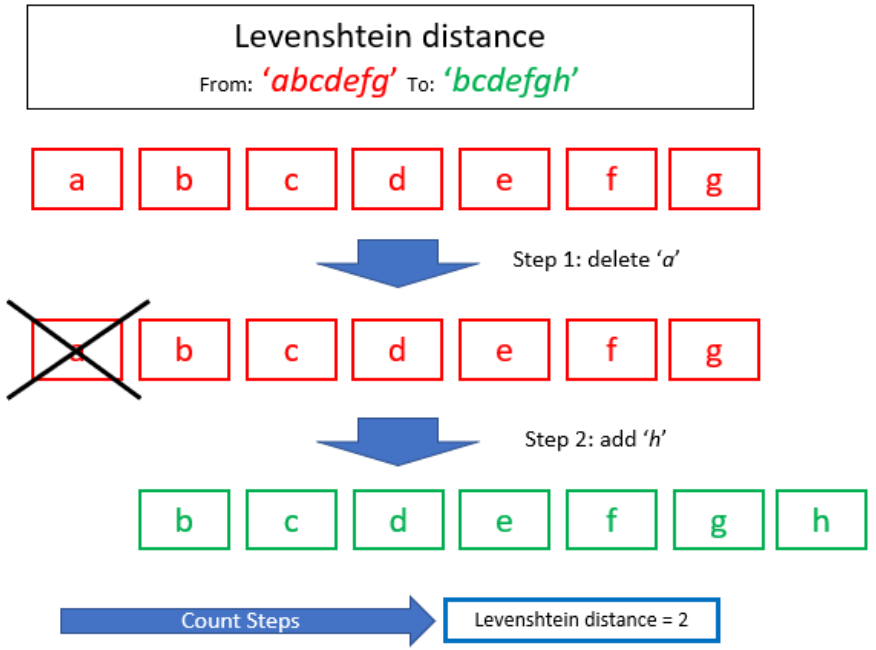
- String similarity
- The number of operations needed to convert one string to another.

Operation types:

- Inserting a character counts as an operation
- Deleting a character counts as an operation
- Substituting a character counts as an operation

Application

- Spelling correction
- Plagiarism detection



https://towardsdatascience.com/3-text-distances-that-every-data-scientist-should-know-7fcdf850e510

Manhattan Distance

Manhattan (City Block) distance is calculating the distance between two points in a grid like path.

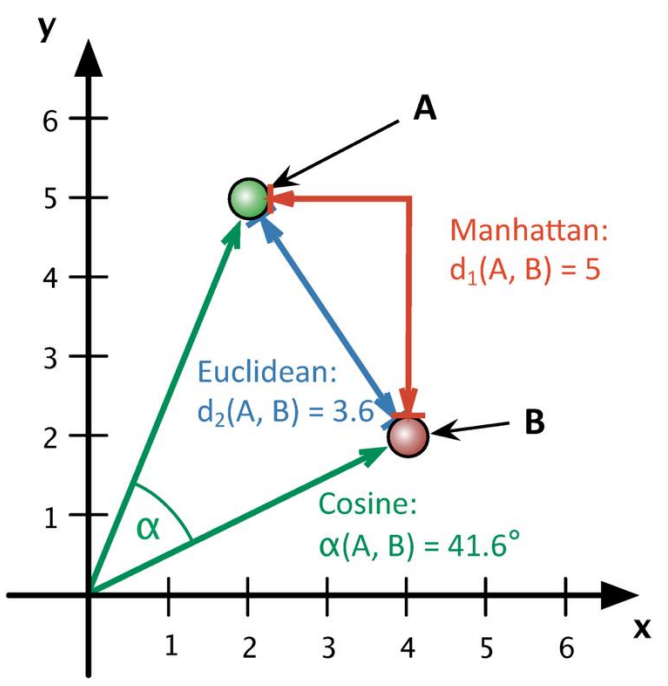
$$d_m = |x_2 - x_1| + |y_2 - y_1|$$

$$d_m = |4 - 2| + |2 - 5|$$

In several machine learning applications, it is important to discriminate between elements that are exactly zero and elements that are small but nonzero. In these cases, we turn to a function that grows at the same rate in all locations, but retains mathematical simplicity: the L1 norm.

Manhattan distance is usually preferred over the more common Euclidean distance when:

- high dimensionality in the data,
- dataset has discrete and/or binary attributes.



Eucledean Distance

Euclidean distance is the shortest between the 2 points irrespective of the dimensions.

$$d_e = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d_e = \sqrt{(4 - 2)^2 + (2 - 5)^2}$$

This metric is commonly used in clustering algorithms such as K-means. Euclidean distance works great when you have low-dimensional data and the magnitude of the vectors is important to be measured. Methods like kNN and HDBSCAN show great results out of the box if Euclidean distance is used on low-dimensional data.

- Sensitive to outliers.

Euclidean distance is the shortest between the 2 points irrespective of the dimensions.

$$d_e = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d_e = \sqrt{(4 - 2)^2 + (2 - 5)^2}$$

This metric is commonly used in clustering algorithms such as K-means. Euclidean distance works great when you have low-dimensional data and the magnitude of the vectors is important to be measured. Methods like kNN and HDBSCAN show great results out of the box if Euclidean distance is used on low-dimensional data.

- Sensitive to outliers.

Cosine Similarity

The cosine similarity is simply the cosine of the angle between two vectors. It also has the same inner product of the vectors if they were normalized to both have length one.

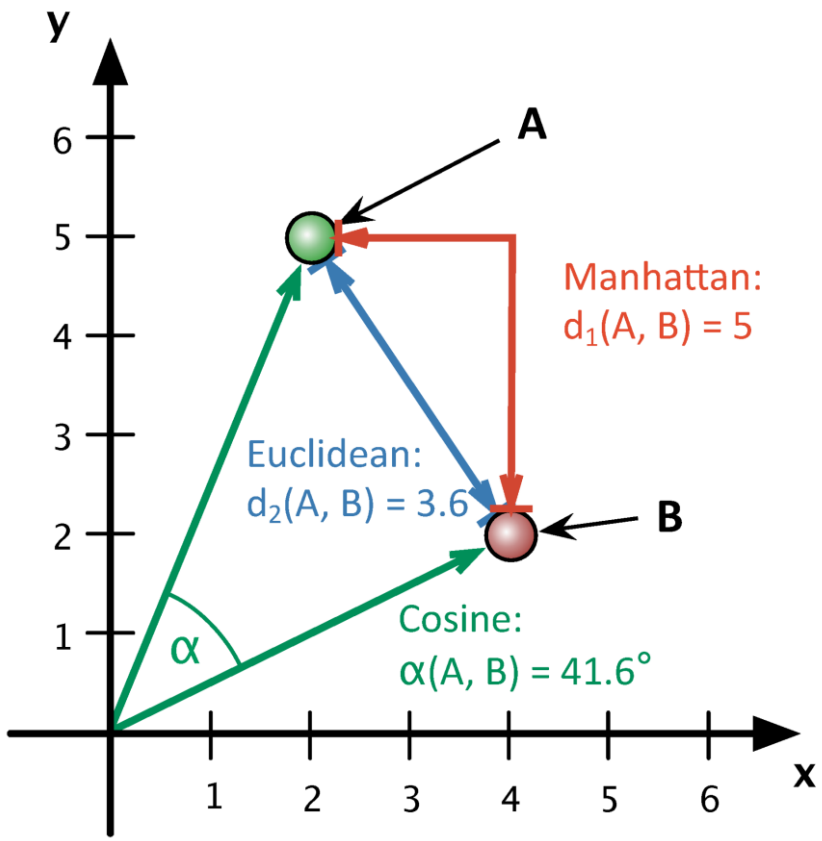
Two vectors with exactly the same orientation have a cosine similarity of 1, whereas two vectors diametrically opposed to each other have a similarity of -1. Note that their magnitude is not of importance as this is a measure of orientation.

$$\cos(\theta) = \frac{x_1 x_2 + y_1 y_2}{\sqrt{x_1{}^2 + y_1{}^2}\sqrt{x_2{}^2 + y_2{}^2}}$$

$$\cos(\theta) = \frac{2*4+5*2}{\sqrt{2^2+5^2}\sqrt{4^2+2^2}} = \frac{18}{5.38*4.47} = 0.748$$

If the angle between the two points is 0 degrees in the above figure, then the cosine similarity, Cos 0 = 1 and Cosine distance is 1- Cos 0 = 0. Then we can interpret that the two points are 100% similar to each other.

$$d_{cos} = 1 - \cos(\theta)$$



Cosine Similarity: Pros and Cons

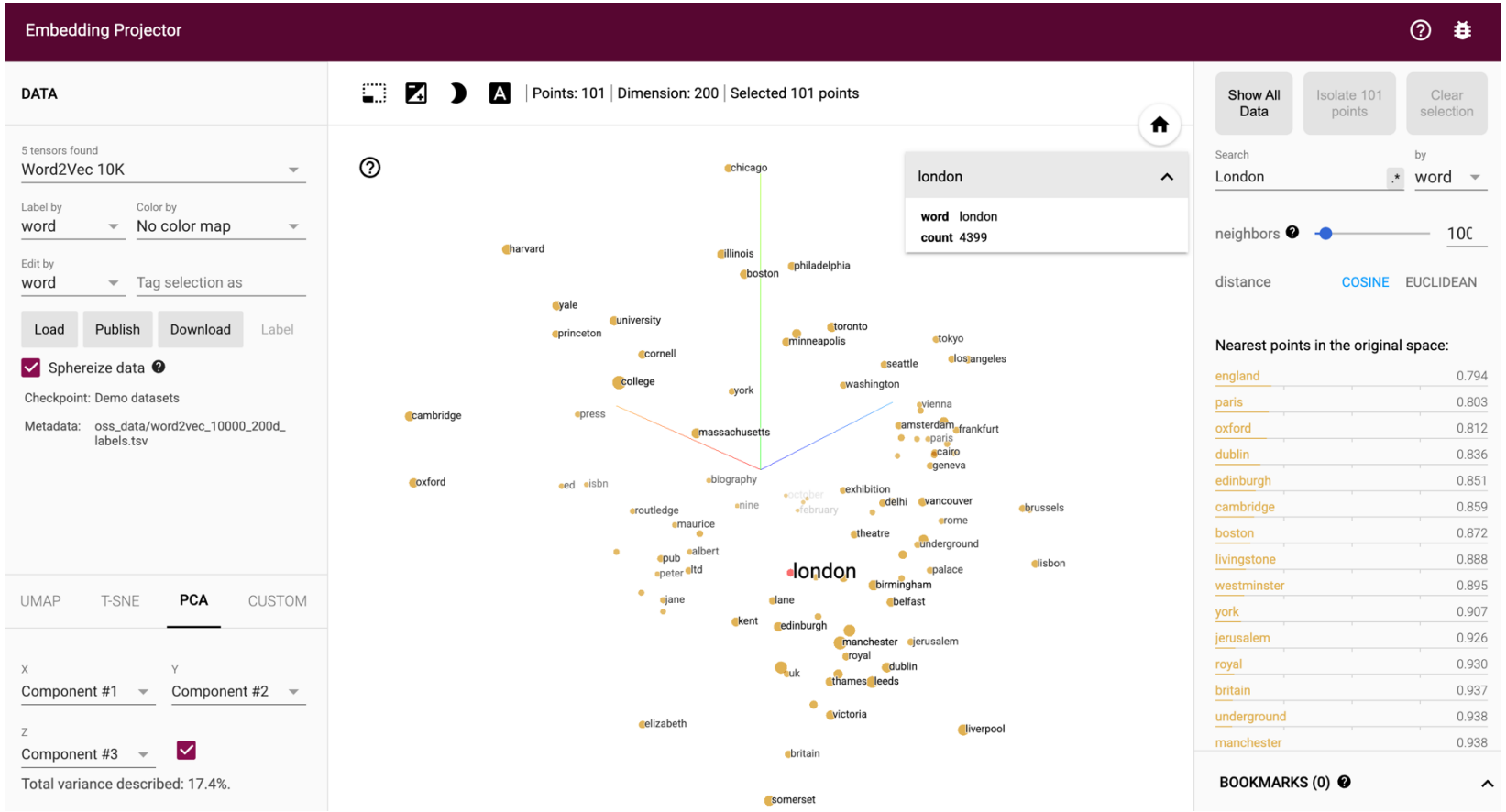
- Cosine similarity has often been used to counteract Euclidean distance's problem with high dimensionality.
- One disadvantage of cosine similarity is that the magnitude of vectors is not considered, merely their direction. In practice, this means that the differences in values are not fully taken into account. If you take a recommender system, for example, then the cosine similarity does not consider the difference in rating scale between different users.

Cosine similarity is useful when we have high-dimensional data and when the magnitude of the vectors is not of importance. It is frequently used when the data is represented by word counts. For example, when a word occurs more frequently in one document over another this does not necessarily mean that one document is more related to that word. It could be the case that documents have uneven lengths and the magnitude of the count is of less importance. Then, we can best be using cosine similarity which disregards magnitude.

Agenda

01	Session goals
02	Semantics
03	Exercise
04	Representation learning
05	Exercise

Embeddings


Embedding Projector
DATA
5 tensors found
Word2Vec 10K
Label by
word
Color by
No color map
Edit by
word
Tag selection as
Load
Publish
Download
Label
Sphereize data
Checkpoint: Demo datasets
Metadata: oss_data/word2vec_10000_200d_labels.tsv
UMAP
T-SNE
PCA
CUSTOM
X
Component #1
Y
Component #2
Z
Component #3
Total variance described: 17.4%.
Points: 101 | Dimension: 200 | Selected 101 points
london
word london
count 4399
chicago
harvard
illinois
philadelphia
boston
yale
university
princeton
cornell
college
york
cambridge
press
massachusetts
oxford
ed
isbn
biography
toronto
minneapolis
tokyo
seattle
los angeles
washington
vienna
amsterdam
frankfurt
paris
cairo
geneva
exhibition
delhi
vancouver
brussels
rome
theatre
underground
palace
lisbon
birmingham
belfast
lane
kent
edinburgh
manchester
jerusalem
royal
dublin
uk
thames
leeds
victoria
liverpool
elizabeth
britain
somerset
routledge
maurice
albert
pub
peter
ltd
jane
nine
october
february
london
Show All Data
Isolate 101 points
Clear selection
Search
London
by
word
neighbors
100
distance
COSINE
EUCLIDEAN
Nearest points in the original space:
england 0.794
paris 0.803
oxford 0.812
dublin 0.836
edinburgh 0.851
cambridge 0.859
boston 0.872
livingstone 0.888
westminster 0.895
york 0.907
jerusalem 0.926
royal 0.930
britain 0.937
underground 0.938
manchester 0.938
BOOKMARKS (0)

Explore & Discuss

Pick your favourite song. Get the lyrics to it and save it line by line into a csv.

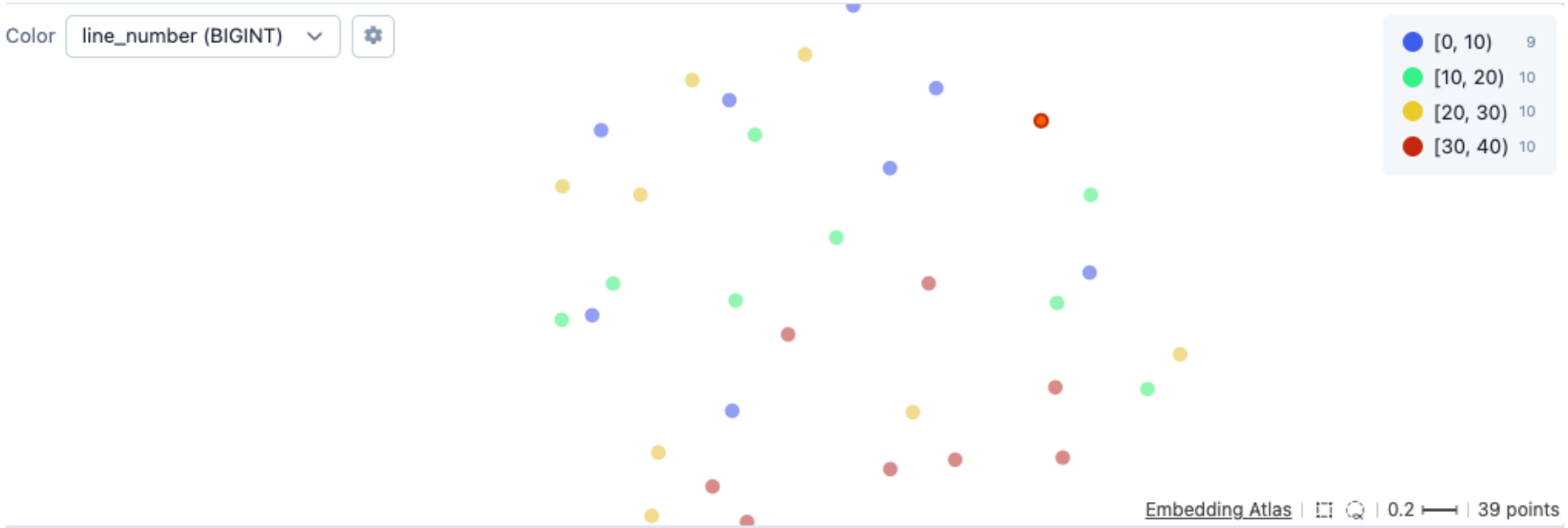
Open the app:
https://apple.github.io/embedding-atlas/app/

Load your song csv into Embedding Atlas. Observe your song.

Model information: **all-MiniLM-L6-v2**

all = trained for general-purpose sentence embedding tasks
MiniLM = a compact transformer model, smaller than BERT-like models
L6 = 6 transformer layers
v2 = version 2 of this sentence-transformer model

Observe Embedding Atlas



Page 1 of 1

song_title	line_number	line	__row_index__	line_proj_x
Punchline	1	I can't help but be destructive right now	0	-3.667
Punchline	2	It's been weeks since I saw your outline	1	-2.731
Punchline	3	In my room is a silence so loud	2	-1.834
Punchline	4	This is what losing hope might sound like	3	-4.444
Punchline	5	Put me down so I'm in my place	4	-1.547
Punchline	6	Say it's over for breathing space	5	-2.949
Punchline	7	All I was was a route to change	6	-4.497

Agenda

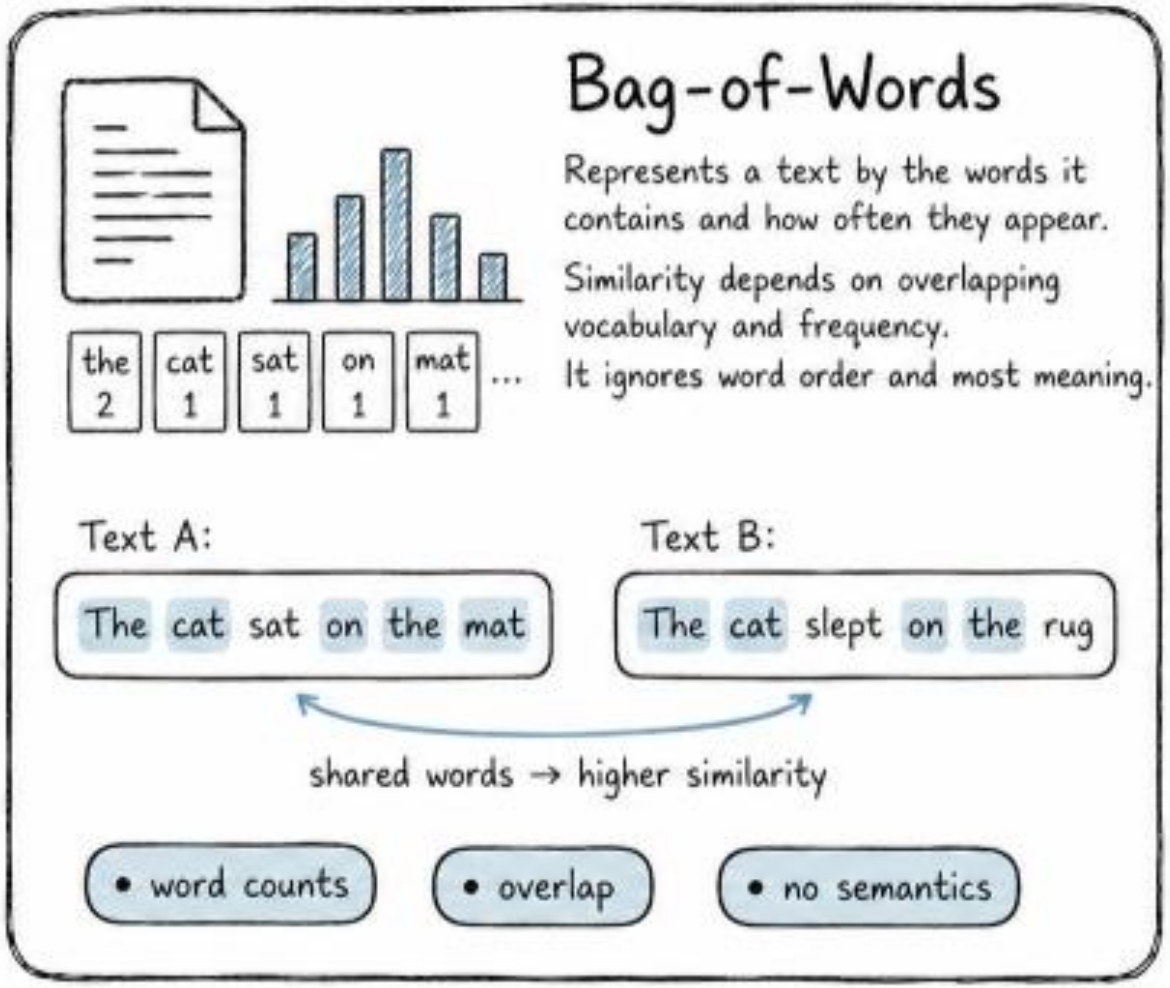
01	Session goals
02	Semantics
03	Exercise
04	Representation learning
05	Exercise

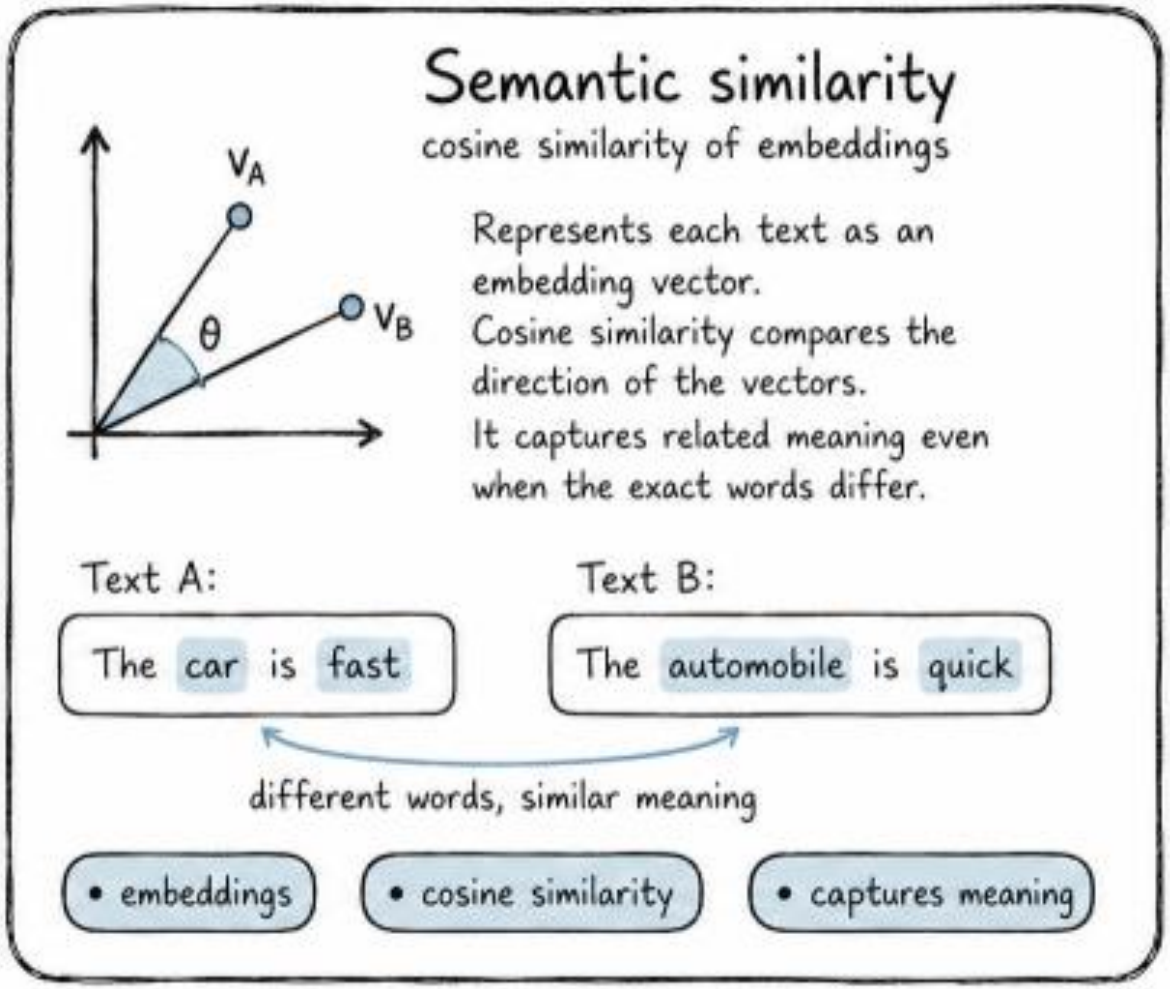
Representation Learning

Embeddings are learned vector **representations** that map discrete or complex data into a continuous numerical space, where geometric proximity reflects semantic, functional, or contextual similarity.

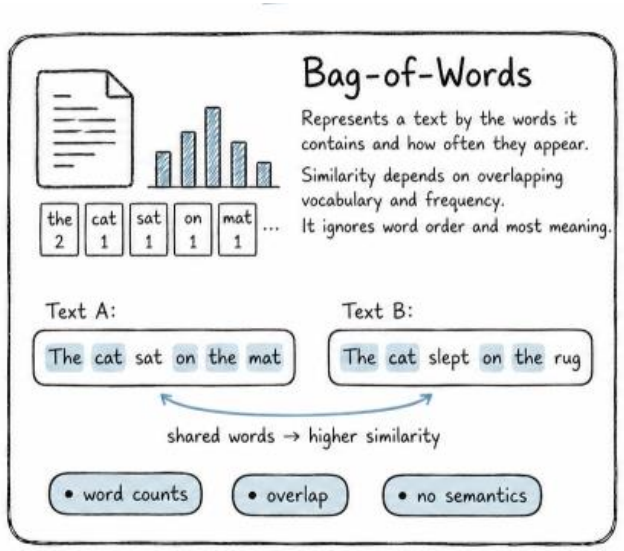
Representation type	Examples	Description
Lexical sparse embeddings	Bag-of-Words, TF-IDF, BM25	Represent text by word occurrence or word importance
Static dense embeddings	Word2Vec, GloVe, FastText	Learn fixed semantic vectors for words
Contextual dense embeddings	BERT-style embeddings, sentence transformers	Meaning depends on context
Document / sentence embeddings	OpenAI embeddings, E5, BGE	Represent whole passages/documents for semantic search
Multimodal embeddings	CLIP, image-text embeddings	Put text, images, etc. into related vector spaces

Measuring Linguistic Similarity





Bag-of-Words



Text A: **“the cat sat on the mat”**
Text B: **“the cat slept on the rug”**
Vocabulary: `[the, cat, sat, on, mat, slept, rug]`
BoW vectors:
Text A: `[2, 1, 1, 1, 1, 0, 0]`
Text B: `[2, 1, 0, 1, 0, 1, 1]`
Cosine similarity:

$$\frac{A \cdot B}{||\ A\ ||||\ B\ ||}$$

Dot product:

$$2 \times 2 + 1 \times 1 + 1 \times 1 = 6$$

Vector lengths:

$$||\ A\ ||= \sqrt{8}, ||\ B\ ||= \sqrt{8}$$

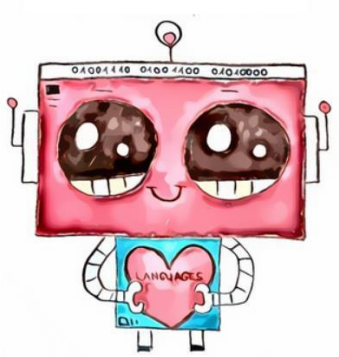
Similarity:

$$\frac{6}{\sqrt{8} \times \sqrt{8}} = \frac{6}{8} = 0.75$$

Bag-of-Words similarity score is 0.75.
This means the two texts are fairly similar **lexically**, because they share several words, even though BoW does not really understand the meaning.

Dimension Reduction

≠



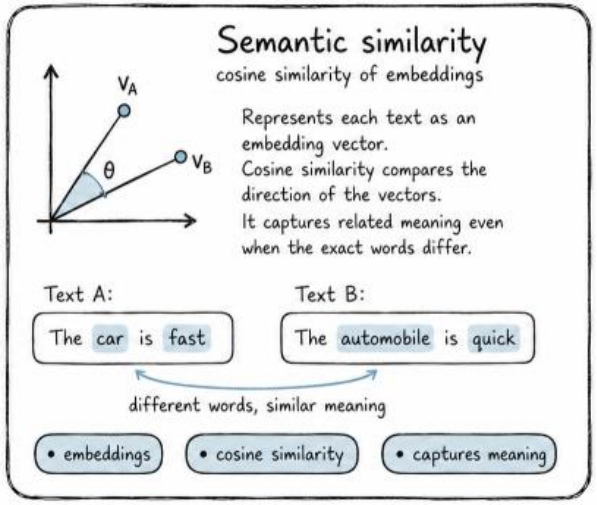
woman = [0, 1,0,0,0]

robot = [1,0,0,0,0]

- The number of dimensions increases linearly as we add words to the vocabulary.
- The embedding matrix is very sparse, mainly zeros.

☞ Reduce vector sizes: such dense vectors are called ‘word embeddings’

Semantic Similarity



Word	Embedding
cat	`[1.0, 0.9]`
sat	`[0.4, 0.7]`
mat	`[0.2, 0.8]`
slept	`[0.3, 0.7]`
rug	`[0.2, 0.85]`

Text A: **“the cat sat on the mat”**
Text B: **“the cat slept on the rug”**
With embeddings, we do **not count shared words directly**. Instead, each word is represented as a vector, and the sentence vector can be approximated by averaging the word vectors.

Sentence embedding for Text A:

$$A = \frac{cat + sat + mat}{3}$$
$$A = [0.53, 0.80]$$

Sentence embedding for Text B:

$$B = \frac{cat + slept + rug}{3}$$
$$B = [0.50, 0.82]$$

Cosine similarity:

$$\frac{A \cdot B}{||\ A\ ||\ ||\ B\ ||}$$
$$\approx 0.999$$

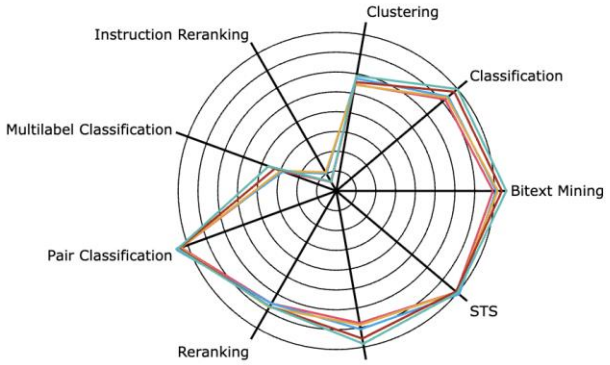
So in this simplified example, the **embed**

Embedding similarity score is about 1.00.
This is high because the sentences describe a very similar situation: **a cat on a soft surface**, even though some words differ. Real embedding models use hundreds or thousands of dimensions, so the exact score would depend on the model.

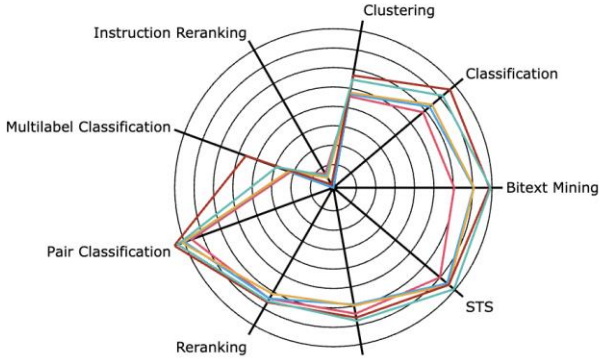
Evaluating Embedding Models

Depends on…

MTEB means **Massive Text Embedding Benchmark**.
It is a benchmark for testing **embedding models**. MTEB checks whether these vectors are useful for real tasks across multiple languages. It is ranking embedding models by how well their vectors work for search, retrieval, similarity, clustering, and related tasks.



All models

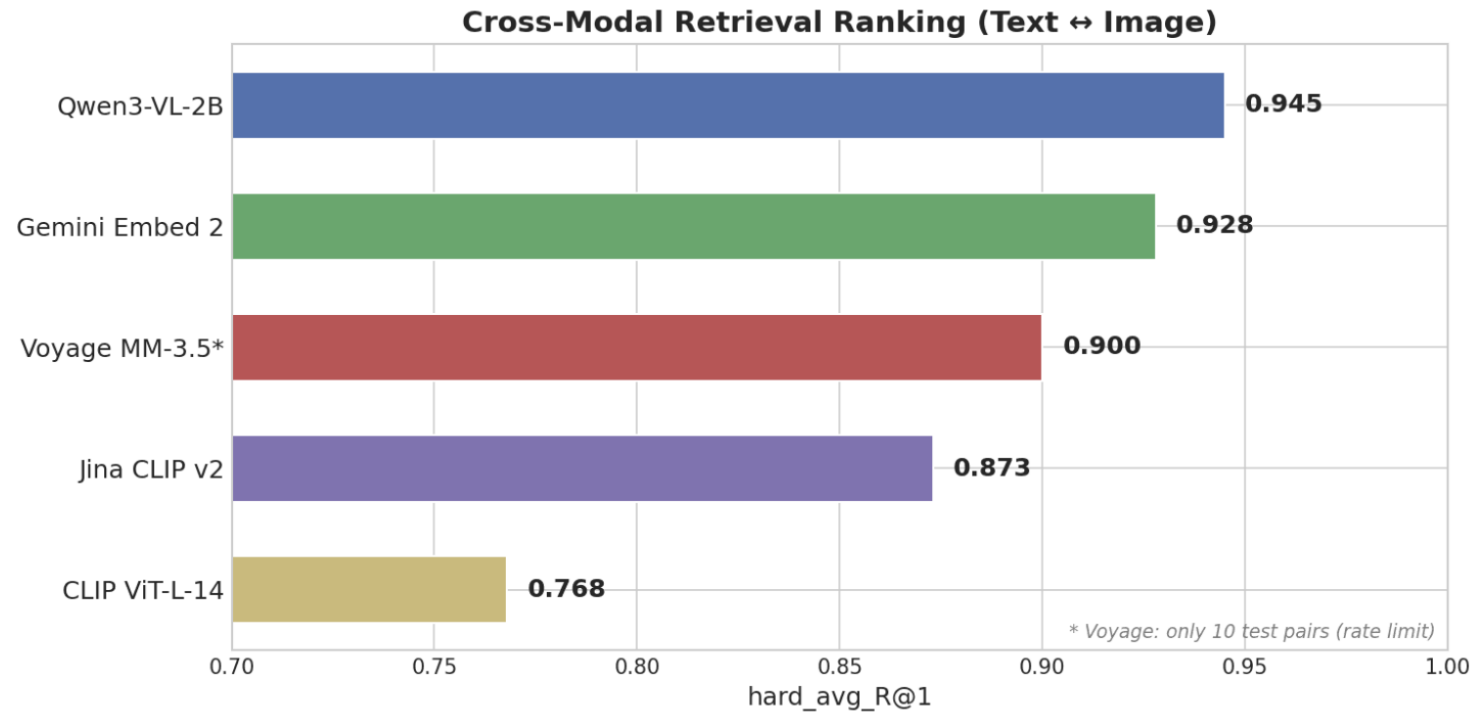


Closed-source

Source: (Multilingual, v2) https://huggingface.co/spaces/mteb/leaderboard

Text to Image

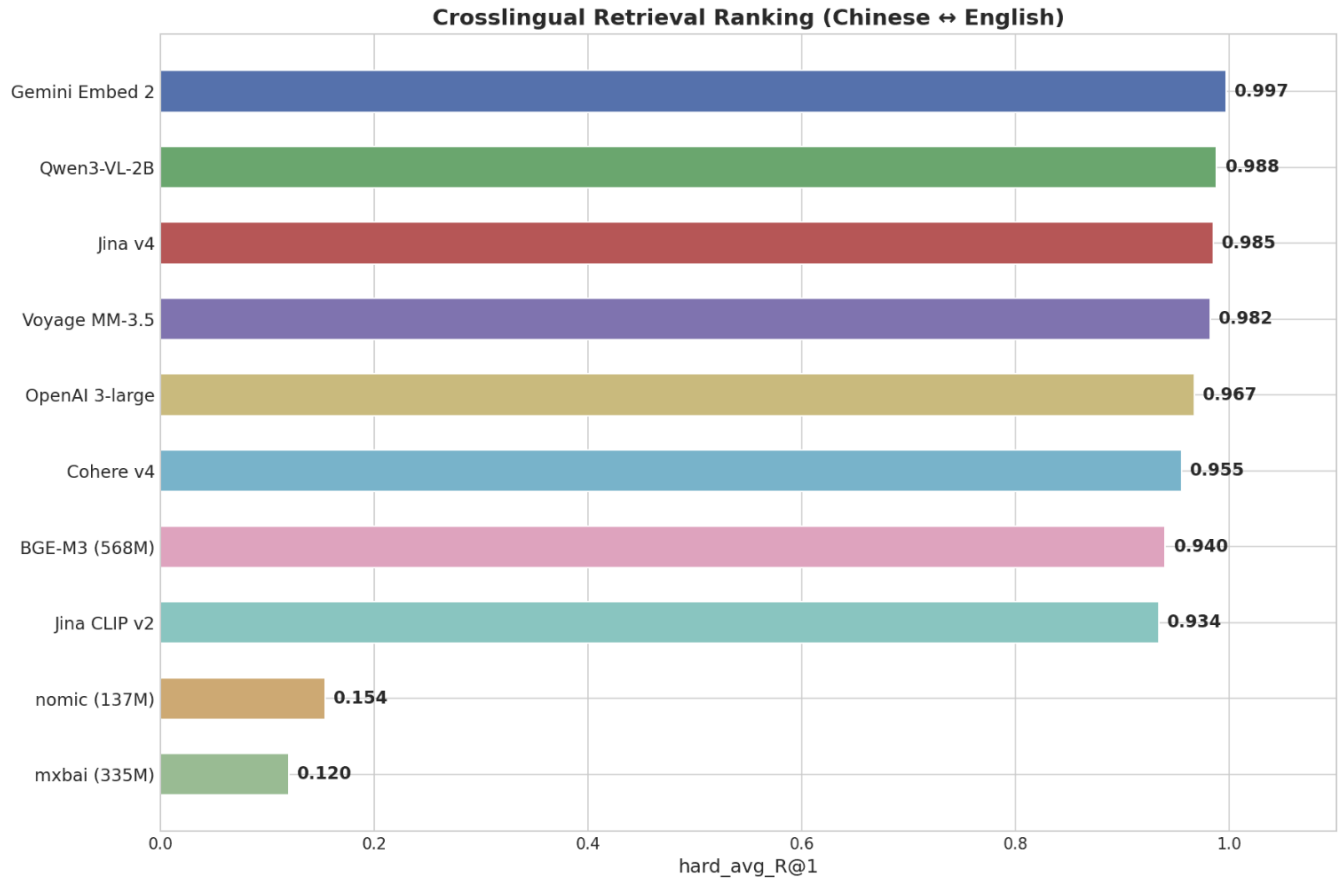
Results



Horizontal bar chart showing Cross-Modal Retrieval Ranking: Qwen3-VL-2B leads at 0.945, followed by Gemini Embed 2 at 0.928, Voyage MM-3.5 at 0.900, Jina CLIP v2 at 0.873, and CLIP ViT-L-14 at 0.768

Source: https://milvus.io/blog/choose-embedding-model-rag-2026.md

Chinese - English



Horizontal bar chart showing Cross-Lingual Retrieval Ranking: Gemini Embed 2 leads at 0.997, followed by Qwen3-VL-2B at 0.988, Jina v4 at 0.985, Voyage MM-3.5 at 0.982, down to mxbai at 0.120

Source: https://milvus.io/blog/choose-embedding-model-rag-2026.md

Agenda

01	Session goals
02	Semantics
03	Exercise
04	Representation learning
05	Exercise

Exercise (BoW)

Pick your favourite song. Get the lyrics to it and save it line by line into a csv.

Open the app:
https://apple.github.io/embedding-atlas/app/

Load your song csv into Embedding Atlas. Observe your song.

Model information: **all-MiniLM-L6-v2**

all = trained for general-purpose sentence embedding tasks
MiniLM = a compact transformer model, smaller than BERT-like models
L6 = 6 transformer layers
v2 = version 2 of this sentence-transformer model

Exercise (Contextual Embeddings)

Pick your favourite song. Get the lyrics to it and save it line by line into a csv.

Open the app:
https://apple.github.io/embedding-atlas/app/

Load your song csv into Embedding Atlas. Observe your song.

Model information: **all-MiniLM-L6-v2**

all = trained for general-purpose sentence embedding tasks
MiniLM = a compact transformer model, smaller than BERT-like models
L6 = 6 transformer layers
v2 = version 2 of this sentence-transformer model

Lyrics Similarity

Benson Boone

Query: “Beautiful Things”

Bag of Words	Semantic similarity
1. Moral of the Story	1. Hold My Hand
2. House of Memories	2. Someone You Loved
3. Mountain Peaks	3. Hold My Girl
4. Play With Fire	4. Love You Twice
5. The Code	5. That’s Us

Semantic results feel like love, vulnerability, longing, and fear of loss. BoW mainly tracks shared lyric vocabulary.

Ed Sheeran

Query: “Punchline”

Bag of Words	Semantic similarity
1. Can’t Help Falling in Love	1. Blurry Eyes
2. Blurry Eyes	2. Ghost Town
3. Break My Heart Again	3. Before You Go
4. Because You Love Me	4. Wrecked
5. My Love	5. Be Alright

Semantic results form a coherent sad-song cluster. BoW can work, but it is fragile when common words dominate.

BoW asks: “Do these songs use the same words?”
Embeddings ask: “Are these songs about similar things?”